Rapport de sécurité — Tropic Line (ShopCart)

Sécuriser une boutique en ligne sur tout son cycle de vie

TEK-UP University — ING-5-SSIR — Module DevSecOps — Enseignant : Oussema Mansouri

Équipe : Nom Prénom 1, Nom Prénom 2, Nom Prénom 3, Nom Prénom 4

Dépôt : `https://github.com/zied458/ShopCart` — Novembre 2026

Résumé. Tropic Line est une boutique de cafés de spécialité (React, Node.js/Express, PostgreSQL) construite avec **huit failles volontaires**, documentées avant toute correction. Une chaîne DevSecOps a ensuite été mise en place : images conteneur minimales et sans root, pipeline GitHub Actions à étape de sécurité bloquante (Gitleaks, Semgrep, Trivy), secrets fournis à l'exécution et révoqués après la fuite, déploiement Kubernetes durci (Pod Security « restricted », NetworkPolicies en refus par défaut, Kyverno), scan dynamique authentifié OWASP ZAP, détection à l'exécution et plan de réponse à incident. Les huit failles sont corrigées, chacune dans son propre commit, et protégées par un test de régression : **13 tests sur 18 échouaient** sur la version vulnérable, **19 sur 19 réussissent** aujourd'hui.

Table des matières

1 Contexte et périmètre **1**

2 Modèle de menace (STRIDE) **1**

3 Failles initiales **2**

4 Cartographie de référence **3**

5 Correctifs **3**

6 Chaîne DevSecOps **4**

6.1 Conteneurs 4

6.2 Pipeline et politique de blocage 5

6.3 Secrets : révoquer d'abord, purger ensuite 5

6.4 Déploiement Kubernetes durci 5

6.5 Test dynamique authentifié 5

6.6 Détection à l'exécution et réponse 6

7 Réflexion critique **6**

A Reproduire **7**

1 Contexte et périmètre

L'application. Une boutique en ligne pour un torréfacteur fictif : six cafés d'origine (Inde, Brésil, Éthiopie…) et du matériel d'extraction. Les clients créent un compte, commandent, suivent leurs commandes et laissent des avis ; un administrateur gère commandes, prix, stocks et clients.

Données sensibles. Comptes (e-mail, empreinte bcrypt du mot de passe, rôle), coordonnées de livraison (nom, téléphone, adresse), historique de commandes, et la clé de signature des jetons de session.

Technologies. Boutique : React 19, TypeScript, Vite, Tailwind CSS, servie par nginx. API : Node.js 24, Express 5, jetons JWT (HS256), bcrypt. Base : PostgreSQL 17 (tables `users`, `products`, `orders`, `order_items`, `reviews`, `security_events`). Livraison : Docker, GitHub Actions, Kubernetes (kind), Kyverno, OWASP ZAP.

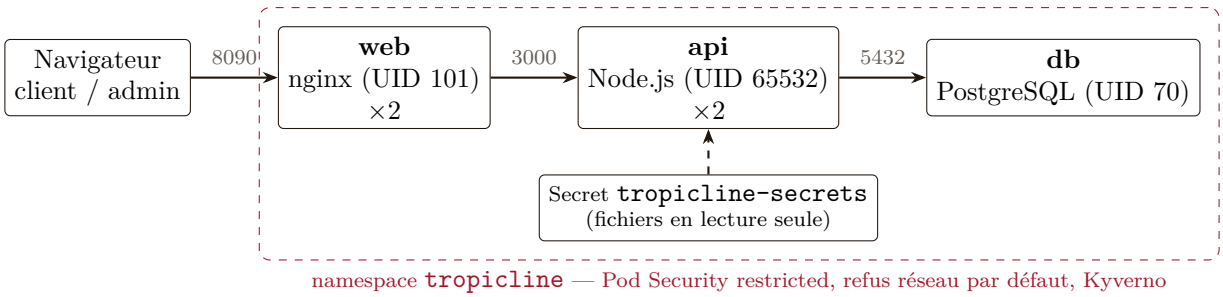


Figure 1 – Architecture déployée et frontières de confiance. Seuls les flux fléchés sont autorisés.

Éthique. Toutes les attaques ont été menées uniquement contre nos propres instances locales (Docker Compose et cluster kind), avec des comptes et des secrets de test.

2 Modèle de menace (STRIDE)

Actifs à protéger : données personnelles des clients ; comptes, en particulier administrateur ; clé de signature des jetons ; intégrité des prix, des stocks et des commandes ; disponibilité de la boutique ; chaîne de livraison (dépôt, pipeline, images, cluster).

Frontières de confiance : navigateur → nginx (Internet, non fiable) ; nginx → API ; API → base ; dépôt → CI → registre → cluster. Tout ce qui vient du navigateur (corps, paramètres, en-têtes, jeton) est contrôlé par l'attaquant.

STRIDE	Menace sur Tropic Line	Faille	Contrôle en place
Spoofing	Fabriquer un jeton admin avec la clé lue dans le dépôt ; deviner un mot de passe	V6	secret à l'exécution, rotation ; verrouillage après 5 échecs
Tampering	Modifier le prix payé ; polluer `Object.prototype` ; image ou action CI altérée	V2, V7	prix lu en base ; champs nommés ; images et actions figées par empreinte, registres autorisés (Kyverno)
Repudiation	Agir sans laisser de trace (aucun journal à l'origine)	A09	journaux JSON corrélés, audit des actions admin
Information disclosure	Injection SQL ; lire la commande d'autrui ; pile d'appels et erreurs SQL renvoyées	V1, V3, V8	requêtes paramétrées ; contrôle de propriété ; erreurs génériques
Denial of service	Épuiser le serveur ; verrouiller le compte d'autrui	—	limites CPU/mémoire, quotas, corps ≤ 100 Ko ; verrouillage par couple adresse + compte
Elevation of privilege	S'inscrire en admin ; script volant la session d'un admin ; sortie du conteneur	V4, V5	rôle fixé et vérifié en base ; rendu texte + CSP ; non-root, sans capacités, lecture seule

Table 1 – Analyse STRIDE : menaces, failles du projet correspondantes et contrôles.

3 Failles initiales

La première version (commit `c70fc4a`) contient huit failles volontaires, chacune marquée `Vn` dans le code et documentée *avant* correction (emplacement, cause, impact) dans `docs/vulnerabilities.md`. Chaque impact a été démontré sur notre instance.

#	Faille	Emplacement	Impact démontré	Gravité
V1	Injection SQL	`/api/products?search=`	une requête `UNION SELECT` renvoie 7 comptes avec e-mail et empreinte du mot de passe	critique
V2	Prix fourni par le client	`POST /api/orders`	moulin à 165 TND acheté 0,001 TND	élevée
V3	IDOR sur les commandes	`GET /api/orders/:id`	un client lit nom, téléphone, adresse d'un autre	élevée
V4	Rôle choisi à l'inscription	`/api/auth/register`	`"role":"admin"` crée un administrateur	critique
V5	XSS stocké	avis produit (API + page)	un script dans un avis s'exécute chez chaque visiteur	élevée
V6	Secret dans le code	`config.js`, Compose	jeton admin fabriqué : 6 fiches clients lues (HTTP 200)	critique
V7	Dépendance vulnérable	lodash 4.17.11	pollution de prototype (CVE-2019-10744) modifiant le serveur	critique
V8	Mauvaise configuration	`app.js`	CORS `*`, erreurs SQL et piles renvoyées, `X-Powered-By`	moyenne

Table 2 – Failles de la version initiale (gravité qualitative : impact × facilité d'exploitation).

4 Cartographie de référence

Chaque faille est rattachée à l'OWASP Top 10 (édition 2025), à la CWE et à STRIDE, avec les contrôles qui l'ont vue.

#	OWASP Top 10 :2025	CWE	STRIDE	Analyse statique (CI)	ZAP (exécution)	Test
V1	A05 Injection	CWE-89	I	Semgrep (règle d'équipe)	**High** confirmé	2
V2	A06 Insecure Design	CWE-602	T	—	—	1
V3	A01 Broken Access Control	CWE-639	I	—	—	2
V4	A01 Broken Access Control	CWE-915, 269	E	Semgrep (règle d'équipe)	—	1
V5	A05 Injection	CWE-79	E	Semgrep (règle d'équipe)	confirmé (persistant)	3
V6	A04 Cryptographic Failures	CWE-321, 798	S	Gitleaks, Semgrep	— (corrigée avant)	—
V7	A03 Supply Chain Failures	CWE-1395, 1321	T	Trivy (dépendances, image)	—	2
V8	A02 Misconfiguration, A10	CWE-209, 942, 693	I	Semgrep (2 avertissements)	confirmé (CORS, en-têtes)	4

Table 3 – Cartographie des failles. « — » : non détectée par ce moyen.

Contrôle	Ce qu'il couvre	Référence
Gitleaks (tout l'historique)	secrets dans le code et l'historique	A04, CWE-798
Semgrep (`p/default` + règles d'équipe)	injection, XSS, rôle lu dans la requête, secrets	A05, A01, A02
Trivy `fs` et `image`	CVE des dépendances et des paquets système, secrets dans l'image	A03, CWE-1395
Trivy `config`, Pod Security, Kyverno	privilèges, root, capacités, registres, limites	A02, « restricted » (PSS)
NetworkPolicies, comptes de service	mouvement latéral, exfiltration, accès à l'API Kubernetes	moindre privilège
ZAP authentifié	injection et configuration à l'exécution	A05, A02
Journaux, règles de détection	attaques en cours, traçabilité	A09

Table 4 – Contrôles de la chaîne DevSecOps et références couvertes.

5 Correctifs

Méthode : (1) écrire un test de régression du point de vue de l'attaquant, exécuté contre l'application déployée ; (2) vérifier qu'il échoue sur la version vulnérable ; (3) corriger la **cause** ; (4) vérifier test, Semgrep, Trivy et ZAP ; (5) un commit par faille. Les tests tournent ensuite dans la CI après chaque déploiement : une faille réintroduite casse le pipeline.

#	Correctif	Preuve
V1	Requête paramétrée (`ILIKE $1`) : le texte est une donnée, jamais du code ; `%` et `_` échappés ; 100 caractères au plus ; tri choisi dans une liste fixe.	tests 2/2 ; Semgrep ; ZAP : High disparu
V2	Prix lu dans `products`, prix du client ignoré (et plus envoyé) ; ligne verrouillée (`FOR UPDATE`) contre la double vente du dernier sachet.	test : commande à 0,001 facturée 18 TND
V3	La commande doit appartenir à l'appelant ; sinon **404**, comme une commande inexistante (ne confirme pas les numéros utilisés).	tests 2/2
V4	Rôle toujours `customer` ; le rôle admin est relu **en base** à chaque requête, pas dans le jeton. Remédiation : 3 comptes déjà élevés rétrogradés.	test : compte créé `customer`, route admin 403
V5	Avis affiché comme **texte** (échappé par React) ; défense en profondeur : CSP `script-src 'self'` sans script en ligne ; 1 000 caractères au plus.	tests 3/3 ; navigateur : aucun script exécuté, aucune violation CSP
V6	Secrets fournis en fichiers à l'exécution, refus de démarrer sans eux ; **révoqués d'abord** (nouvelle clé, nouveau mot de passe) ; purge de l'historique préparée.	jeton forgé : 200 → 401
V7	lodash **supprimé** ; la mise à jour du profil copie des champs nommés, type et longueur vérifiés.	test de pollution ; Trivy : 1 critique + 3 élevées → 0
V8	CORS supprimé (même origine via nginx), `X-Powered-By` désactivé, en-têtes de sécurité, erreurs génériques avec identifiant de corrélation, détail dans le journal seulement.	tests 4/4 ; ZAP : 9 alertes disparues

Table 5 – Correctifs (un commit par faille) et preuves.

Choix notables. Pour V1, filtrer les apostrophes ou le mot `UNION` se contourne et casse des recherches légitimes ; la requête paramétrée supprime la cause. Pour V3, répondre 404 plutôt que 403 évite de révéler quels numéros existent. Pour V7, mettre lodash à jour aurait corrigé les CVE connues, mais fusionner tout un corps de requête reste une affectation de masse ; supprimer la dépendance retire aussi une surface d'attaque. Pour V4, corriger le code ne suffit pas : les comptes déjà élevés ont été rétrogradés.

Contrôle	Version vulnérable	Version corrigée
Tests de régression	13 échecs sur 18	**19 / 19** (lockout inclus)
Semgrep	4 `ERROR` (V1, V4, V5, V6)	**0 ERROR** (2 avertissements, faux positifs)
Trivy, dépendances de l'API	1 critique, 3 élevées	**0**
ZAP authentifié	1 High, 5 Medium, 3 Low	**0 High réelle**, 11 alertes sur 13 disparues
Gitleaks	3 secrets	**0** (révoqués, puis historique purgé)

Table 6 – Avant / après.

6 Chaîne DevSecOps

6.1 Conteneurs

Images multi-étapes ; API sur base *distroless* (ni shell ni gestionnaire de paquets), utilisateur 65532 ; boutique sur nginx non privilégié, utilisateur 101, paquets mis à jour et `curl` retiré. Images de base figées par empreinte, `.dockerignore` en liste blanche (pas de `.git`, pas de secret), code appartenant à root donc non modifiable par le processus. Exécution en lecture seule, sans capacité Linux, `no-new-privileges`.

6.2 Pipeline et politique de blocage

GitHub Actions, outils dans des images à version figée et actions figées par empreinte de commit. Chaque barrière publie un tableau lisible sur la page de l'exécution, et un job final les résume.

Barrière	Bloque sur	Justification
Secrets (Gitleaks)	toute détection, tout l'historique	un secret poussé est compromis ; valeurs caviardées
Code (Semgrep)	`ERROR`	failles exploitables ; `WARNING` trop dépendant du contexte
Dépendances, images (Trivy)	`HIGH/CRITICAL` avec correctif ; image en root	corriger coûte une mise à jour ; sans correctif, bloquer figerait tout
Fichiers de déploiement	`HIGH/CRITICAL`	privilèges menant au nœud
Déploiement de test	22 contrôles de durcissement	contrôles binaires
Tests, ZAP, détection	un test, une alerte `High`, une règle muette	preuve à l'exécution

Table 7 – Politique de blocage (détail et exceptions datées : `docs/pipeline-policy.md`).

Le premier passage sur GitHub a échoué comme prévu (secrets, code, dépendances, image API) et n'a rien publié. Une exception n'est acceptée qu'avec une justification, un responsable et une échéance ; un vrai secret n'est jamais mis en exception.

6.3 Secrets : révoquer d'abord, purger ensuite

Les secrets sont des fichiers montés (secrets Docker, Secret Kubernetes en lecture seule, clé par clé), générés aléatoirement au déploiement ; l'API refuse de démarrer sans eux. Après la fuite V6 : (1) **révocation** — nouveau mot de passe (`ALTER USER`), nouvelle clé de signature, redémarrage : le jeton forgé passe de **200 à 401**, l'ancien mot de passe est refusé, les clients se reconnectent ; (2) **purge** — `git filter-repo` remplace les valeurs dans tous les commits d'un miroir, vérifié par Gitleaks (**3** →

0), puis envoi forcé : l'étape « Secrets » du pipeline repasse au vert. Purger sans révoquer laisserait le secret valide dans tous les clones déjà faits.

6.4 Déploiement Kubernetes durci

Namespace en Pod Security « restricted » ; un compte de service par composant, **aucun droit** sur l'API Kubernetes et aucun jeton monté ; `securityContext` : non-root, lecture seule, sans capacité, seccomp, limites ; NetworkPolicies en **refus par défaut** (web → api → db, DNS seulement si nécessaire, aucune sortie vers Internet) ; quatre politiques Kyverno en mode `Deny` (durcissement, limites, registres et versions explicites, secrets en fichiers). `verify.sh` prouve **22 contrôles sur le cluster en marche** : par exemple, un pod root ou une image `nginx:latest` est refusé, la boutique ne joint pas la base, l'API ne joint pas Internet.

6.5 Test dynamique authentifié

ZAP 2.17 se connecte comme client et envoie le jeton sur chaque requête (preuve : `/api/auth/me` doit répondre 200), importe la description OpenAPI de l'API et explore la boutique. Avant correction, il confirme **V1** (injection SQL, High), **V5** et **V8**. Après correction, il ne reste aucune alerte High réelle ; un faux positif « Path Traversal » (confiance faible) est déclaré pour une seule URL et un seul paramètre, avec sa raison.

6.6 Détection à l'exécution et réponse

L'API écrit une ligne JSON par requête et par événement de sécurité, avec l'identifiant de corrélation de nginx ; e-mails masqués, ni mot de passe ni jeton. Trois règles sur 5 minutes : **brute force** (5 échecs sur un compte depuis une adresse → verrouillage 429), **sonde d'injection** (3 motifs SQL ou script), **sonde d'accès** (10 refus ou commandes inconnues). Les compteurs sont partagés en base entre les répliques ; des métriques Prometheus restent internes. La CI vérifie que les attaques du job (tests, ZAP) déclenchent les trois règles. Le plan de réponse (`docs/incident-response.md`) définit rôles, gravités P1–P3 et cinq scénarios avec commandes, dont la mise en quarantaine d'un pod ; il a été exercé sur le cluster.

7 Réflexion critique

Ce que les outils voient, et ce qu'ils ne voient pas.

— Les règles publiques de Semgrep n'ont trouvé **aucune** des huit failles ; nos règles d'équipe en trouvent quatre, mais elles ont été écrites en connaissant les failles : ce biais surestime l'analyse statique. Sur un vrai projet, il faut des règles écrites avant de connaître les bugs.

— V2 et V3 sont invisibles pour l'analyse statique *et* pour ZAP : ce sont des failles de logique métier et de contrôle d'accès. Seuls des tests qui connaissent les règles de l'application les couvrent. Le DAST complète le SAST, il ne remplace ni l'un ni l'autre la revue et les tests.

— Les outils produisent aussi du bruit : deux avertissements Semgrep et une alerte ZAP High étaient des faux positifs. Bloquer sans tri pousserait l'équipe à désactiver les contrôles.

Erreurs commises, et ce qu'elles ont appris.

— Un fichier `trivy.yaml` ajouté pour accélérer les scans locaux excluait `node_modules` *aussi* dans le scan d'image : l'image de l'API passait avec la faille critique de lodash. Repéré en vérifiant un résultat trop beau, corrigé. Leçon : un contrôle qui passe doit aussi être vérifié.

— Les premiers compteurs de détection étaient en mémoire : avec deux répliques, trois sondes réparties 2 + 1 ne déclenchaient rien. Ils sont maintenant partagés en base.

— Le test V3 choisissait « la commande d'un autre » d'après l'e-mail saisi ; ZAP avait créé des commandes avec des e-mails fantaisistes, et le test a échoué à tort. Il compare maintenant les comptes propriétaires.
— Le verrouillage a bloqué l'administrateur pendant une démonstration : c'est son rôle, mais la procédure de déblocage a été ajoutée au plan de réponse.

Limites restantes.

— L'historique a été purgé et réécrit (Gitleaks : 0), mais les anciennes valeurs ont été publiques un temps : des copies peuvent subsister (clones, caches). C'est pourquoi la révocation passait d'abord.
— Environnement de test : pas de TLS, Secret Kubernetes non chiffré dans etcd, cluster d'un nœud ; sur kind, le trafic arrive avec l'adresse du nœud, ce qui affaiblit les règles par adresse.
— Pas de centralisation des journaux (SIEM) ni de tableau de bord ; pas de détection au niveau des conteneurs (Falco) ; images non signées (cosign) ; feuille Google Fonts sans empreinte SRI.

Si c'était à refaire. Écrire le modèle de menace et les tests de régression *avant* le code ; intégrer les tests métier au pipeline dès le premier jour ; ajouter TLS, la signature des images, le chiffrement des secrets au repos et un SIEM.

Déclaration d'usage d'outils d'IA. Un assistant d'IA (Claude, Anthropic, via Claude Code) a été utilisé pour produire une partie du code, des configurations, des scripts, de la documentation et de ce rapport. Chaque résultat a été exécuté et vérifié par l'équipe sur ses propres instances (tests, scanners, captures du pipeline) ; l'équipe reste responsable des choix et du contenu.

A Reproduire

```
bash scripts/new-secrets.sh && docker compose up -d --build   # http://localhost:8081
bash scripts/k8s/up.sh && bash scripts/k8s/verify.sh           # cluster durci, 22 controles
BASE_URL=http://127.0.0.1:8090 npm --prefix backend run test:security
bash scripts/zap-scan.sh && bash scripts/demo/attack.sh       # ZAP, puis detection
```

Documents détaillés, dans `docs/` : *vulnerabilities*, *corrections*, *pipeline-policy*, *secrets*, *kubernetes*, *dast*, *detection* et *incident-response* (fichiers `.md`).